

Let's name them *AddService*, *FindService*, *SearchService* and *JournalService*. Refer to Figure 2.

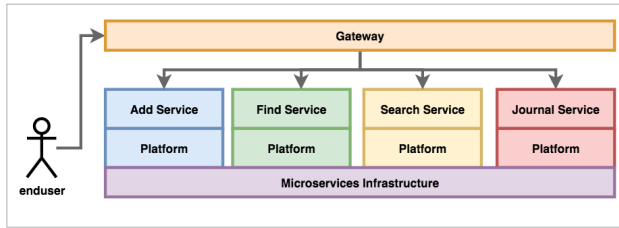


Figure 2: UMS as a set of microservices

Once the UMS is decomposed into the above four smaller microservices, each of them can be developed on different platforms in case such a polyglot environment is beneficial. For instance, *AddService* may be developed using Spring Boot on Java platform, *FindService* may be developed using Express on Node platform, and *SearchService* may be developed using Flask on Python platform.

The point worth noting is that these services are developed independently, even if all of them are developed on the same platform. The *AddService* might be released first, followed by the *FindService*. They do not share any codebase. Code reusability takes the back seat and service independence occupies the driving seat!

## AddService

It's time to design the microservice for adding users into the system. The design still sticks to the fundamental principles of object-oriented design. In other words, we will follow the SOLID principles and use patterns like factories, adapters, etc.

By taking inspiration from the Onion model, which we have seen in the past as part of domain-driven design, *AddService* is also modelled on the lines of domain layer, application layer and infrastructure layer. The domain layer consists of only domain objects. The application layer acts as the gateway to the domain layer. And the infrastructure layer offers all the generic services.

## The domain model

This layer primarily consists of entities, value objects and repositories. The domain model is presented in Figure 3.

As explored in the previous part, an entity is a domain object that is uniquely identifiable and mutable. What else other than the user can be an entity in the domain of *AddService*? Though the name of a user is not unique in the real world, for the sake of simplicity, let us treat it as the identity of the user. The identity must be immutable, but the rest of the entity can be mutable. For instance, the phone number of the user can be changed at any time.

Unlike the entities, value objects do not have any unique identity and are comparable only by their values. Usually, the

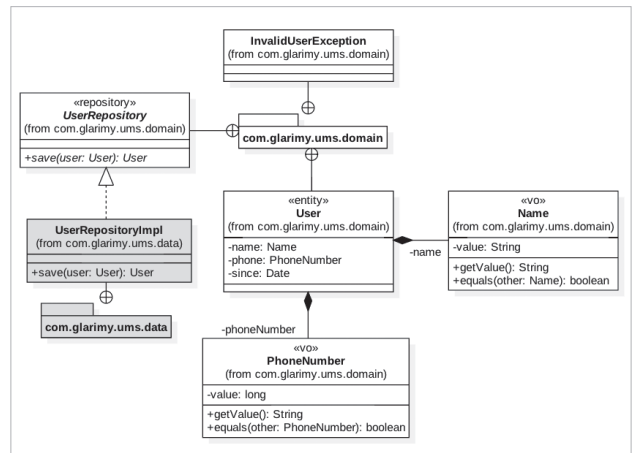


Figure 3: The domain model of *AddService*

primitive types with some sort of constraints are modelled as value objects. For example, the name of the user is modelled as a value object *Name*, since it is a string with some constraints like minimum length, maximum length, accepted characters, etc. Similarly, the phone number also can be modelled as a value object *PhoneNumber*, since it is a long number with constraints like length, positivity, etc.

Since the value objects are immutable, they do not offer any way to change their internals. In other words, they only offer getters, but not any setters.

A repository acts as if it is a cache of aggregates of domain objects. The *UserRepository* is one such repository that saves the user objects. Though the repository belongs to the domain model, the implementation is technology-specific. It may be implemented around MongoDB, MySQL, Cassandra or any other data storage technology. Since the domain does not depend on the technology, the repository implementation *UserRepositoryImpl* is kept out of the domain model.

The following is the implementation of the domain model on Java platform. The *Name*, being a value object, offers only a constructor and getters. The constructor may validate the input parameter for constraints before accepting. In case of an invalid parameter, an exception may be thrown. Also, since a value object is only compared with other objects by their values, the *hashCode()* and *equals()* methods are required to be implemented.

```
public class Name {
    private String value;

    public Name(String value) {
        //TODO: validation
        this.value = value;
    }

    public String getValue() {
```